

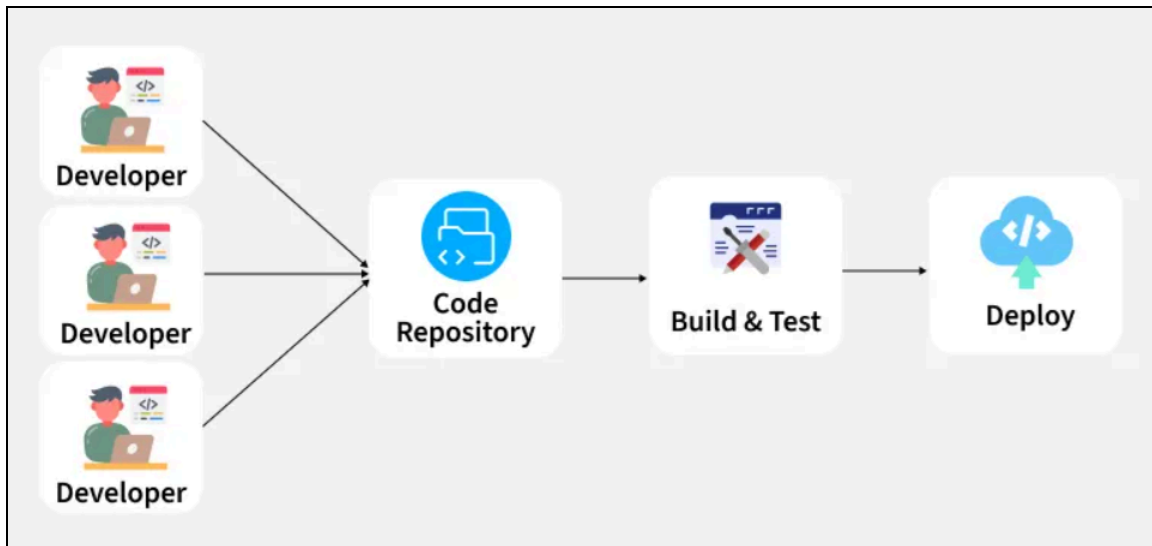
Assignment No. 3

Aim: Study Jenkins Architecture and Plugins with Jenkins Installation and Setup.

Objective: To study the architecture of Jenkins, understand the role and usage of Jenkins plugins, and perform the installation and basic setup of Jenkins in order to gain practical knowledge of Continuous Integration and Continuous Deployment (CI/CD) concepts.

Theory

1. Continuous Integration and Continuous Deployment



CI/CD is an automated workflow that replaces manual steps with pipelines that build, test, and deploy software reliably. CI/CD falls under DevOps (the joining of development and operations teams) and combines the practices of continuous integration and continuous delivery. CI/CD automates much or all of the manual human intervention traditionally needed to get new code from a commit into production, encompassing the build, test (including integration tests, unit tests, and regression tests), and deploy phases, as well as infrastructure provisioning. With a CI/CD pipeline, development teams can make changes to code that are then automatically tested and pushed out for delivery and deployment.

2. What is Jenkins?

Jenkins is an open-source tool that automates various stages of the software development lifecycle. It supports building, testing, and deploying applications across multiple environments, making it a critical component in CI/CD pipelines.

- **Automation:** Automates repetitive tasks such as builds, tests, and deployments.
- **Extensibility:** Over 1,800+ plugins to integrate with various tools like Docker, Kubernetes, Selenium, and Git.
- **Technology Agnostic:** Supports multiple programming languages and tech stacks,

including Java, Python, JavaScript, and more.

- **Ease of Setup:** Provides a user-friendly web-based GUI for configuration.
- **Master-Slave Architecture:** Enables distributed builds for improved performance and scalability.

3. Jenkins Architecture

Jenkins follows a master-slave architecture, where the Jenkins server (master) manages build jobs and delegates their execution to agent nodes (slaves). This setup ensures efficient resource utilization and parallel execution.

1. Jenkins Controller (Master)

- The controller manages the Jenkins dashboard and user interface.
- It schedules jobs, manages plugins, and assigns tasks to agents.
- It does not usually perform heavy build work (to avoid overload).

2. Jenkins Agent (Node)

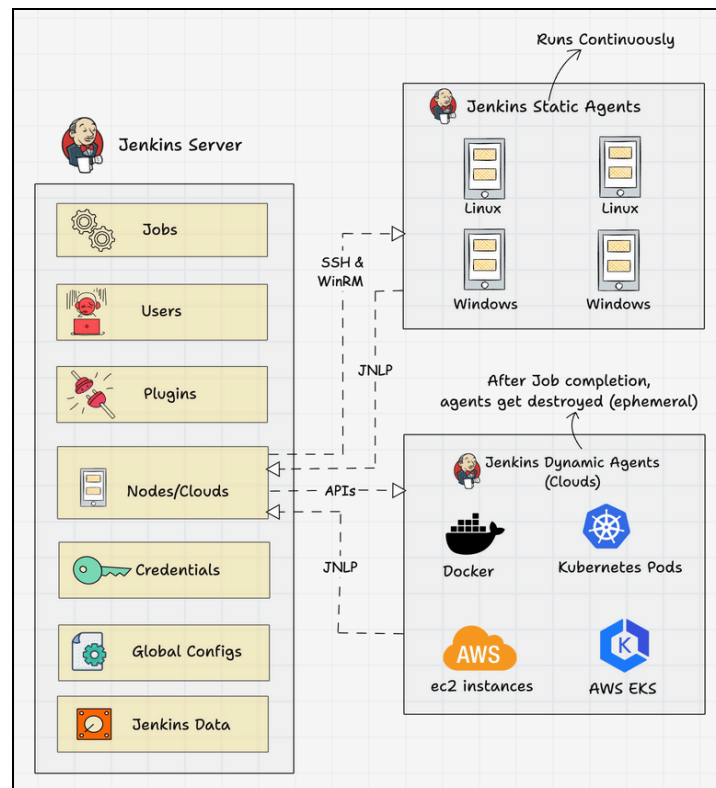
- Agents are machines that actually execute build, test, and deployment tasks.
- Multiple agents can be connected to one controller.
- Agents help run jobs in parallel and support different operating systems or environments.

3. Jenkins Jobs / Pipelines

- Jobs define what tasks Jenkins should perform (build, test, deploy).
- Pipelines describe these steps in code (Jenkinsfile).

4. Plugins

- Plugins extend Jenkins functionality (Git, Maven, Docker, etc.).
- Jenkins depends heavily on plugins to support various tools.

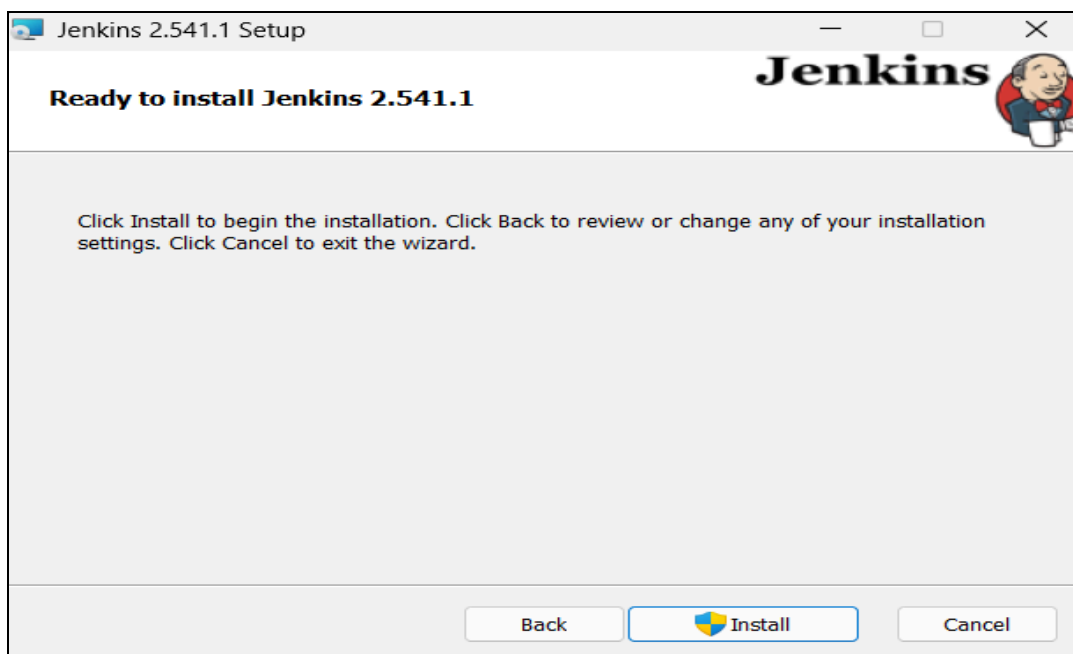


5. Source Code Repository

- Jenkins pulls code from repositories like GitHub or GitLab.
- Any change in code can automatically trigger Jenkins jobs.

Procedure for Jenkins Installation :

1. Install Java JDK on the system and set up environmental variables.
2. Download Jenkins from the official Jenkins website.
3. Install Jenkins and start the Jenkins service.



Jenkins 2.541.1 Setup

Port Selection

Choose a port for the service.

Please choose a port.

Port Number (1-65535):



It is recommended that you accept the selected default port.

4. Access Jenkins through a web browser using <http://localhost:8080>.
5. Unlock Jenkins using the administrator password.

Jenkins 2.541.1 Setup

Service Logon Credentials

Enter service credentials for the service.

Jenkins 2.541.1 installs and runs as an independent Windows service. To operate in this manner, you must supply the user account credentials for Jenkins 2.541.1 to run successfully.

Logon Type:

☒ Run service as LocalSystem (not recommended)

☐ Run service as local or domain user:

Account:

Password:

Getting Started

Unlock Jenkins

To ensure Jenkins is securely set up by the administrator, a password has been written to the log (not sure where to find it?) and this file on the server:

```
C:\ProgramData\Jenkins\.jenkins\secrets\initialAdminPassword
```

Please copy the password from either location and paste it below.

Administrator password

.....

Continue

6. Install suggested plugins.

Getting Started

Customize Jenkins

Plugins extend Jenkins with additional features to support many different needs.

Install suggested plugins

Install plugins the Jenkins community finds most useful.

Select plugins to install

Select and install plugins most suitable for your needs.



7. Create an admin user and complete setup.

Getting Started

Create First Admin User

Username

Rutuja_Bobade

Password

.....

Confirm password

.....

Full name

Rutuja Vishwas Bobade

E-mail address

bobaderutuja13@gmail.com

Jenkins 2.541.1

[Skip and continue as admin](#) [Save and Continue](#)

Getting Started

Instance Configuration

Jenkins URL:

http://localhost:8080/

The Jenkins URL is used to provide the root URL for absolute links to various Jenkins resources. That means this value is required for proper operation of many Jenkins features including email notifications, PR status updates, and the BUILD_URL environment variable provided to build steps.

The proposed default value shown is **not saved yet** and is generated from the current request, if possible. The best practice is to set this value to the URL that users are expected to use. This will avoid confusion when sharing or viewing links.

Jenkins 2.541.1

[Not now](#) [Save and Finish](#)

Getting Started

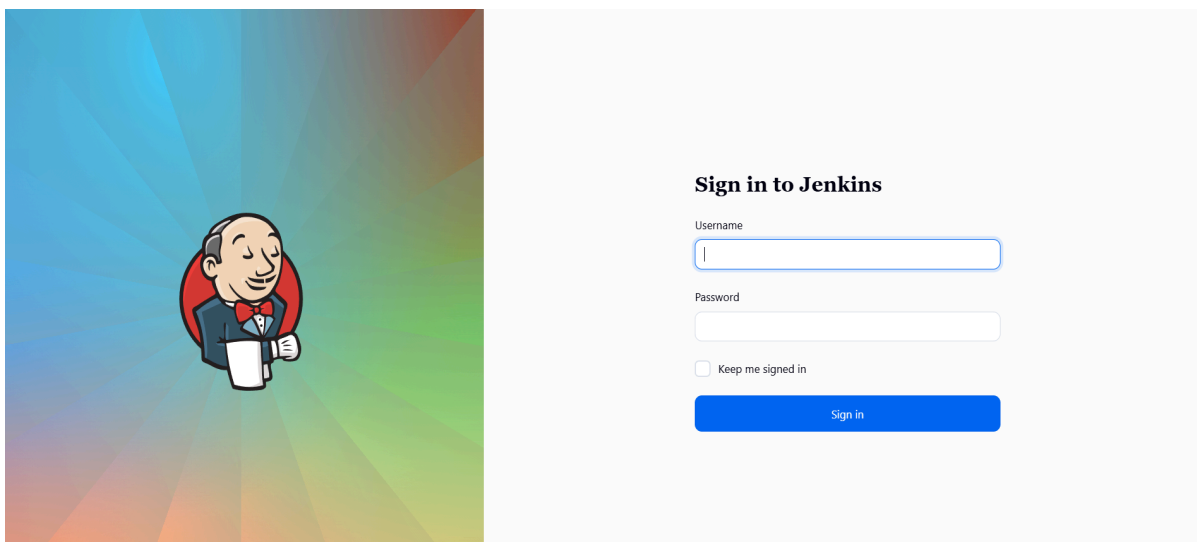
Getting Started

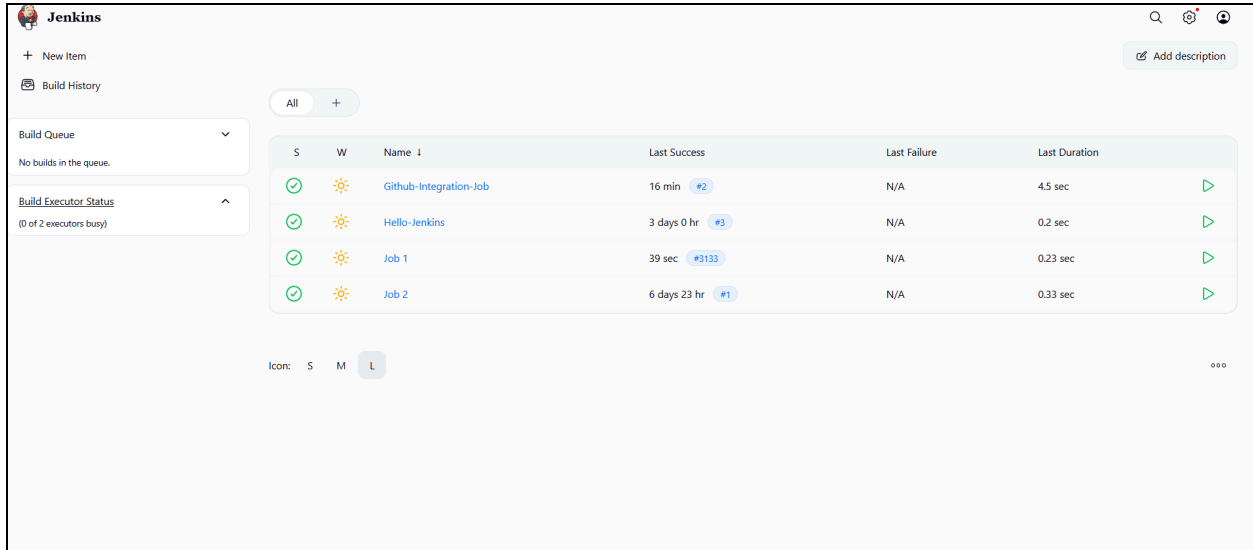
 Folders	 OWASP Markup Formatter	 Build Timeout	 Credentials Binding	** commons-lang3 v3.x Jenkins API ** Ionicons API Folders
 Timestamper	 Workspace Cleanup	 Ant	 Gradle	
 Pipeline	 GitHub Branch Source	 Pipeline: GitHub Groovy Libraries	 Pipeline Graph View	
 Git	 SSH Build Agents	 Matrix Authorization Strategy	 LDAP	
 Email Extension	 Mailer	 Dark Theme		

** - required dependency

Jenkins 2.541.1

8. Create a new Jenkins job and execute a sample build.





Questions

1. What is the difference between (CI/CD) Continuous Delivery and Continuous Deployment?

a. CI (Continuous Integration)

1. CI is the process of automatically building and testing code whenever developers push changes to a shared repository.
2. Its main goal is to detect bugs early by frequently integrating code and running automated tests.
3. CI helps ensure that the codebase always stays stable.

b. CD (Continuous Delivery / Continuous Deployment)

1. CD is the process of automatically preparing or deploying tested code to production or staging environments.
2. Its main goal is to release software faster and more reliably after CI succeeds.
3. With Continuous Delivery, deployment is manual, while with Continuous Deployment, it happens automatically.

2. What are the different types of jobs available in Jenkins?

1. Freestyle Project
 - a. This is the most basic type of Jenkins job.
 - b. It allows you to configure build steps manually using the Jenkins UI.
 - c. It is mainly used for simple build, test, or deployment tasks.
2. Pipeline
 - a. Pipeline jobs define the entire CI/CD process as a code using a Jenkinsfile.
 - b. It supports multiple stages like build, test, and deploy.
3. Multibranch Pipeline
 - a. This job automatically creates pipelines for each branch in a repository.
 - b. It is useful when working with feature branches in Git.

- c. Each branch can have its own Jenkinsfile.
- 4. Maven Project
 - a. This job type is specifically designed for Maven-based Java projects.
 - b. Jenkins handles build and dependency management using Maven.
- 5. Folder
 - a. Folder jobs help organize multiple Jenkins jobs in a structured way
 - b. Useful for large project with many pipelines.
- 6. External Job
 - a. This is used when build processes run outside Jenkins.
 - b. Jenkins only monitors and reports their status.

3. Explain Freestyle job in Jenkins.

A Freestyle Job is the simplest and most basic type of job in Jenkins. It allows users to configure build, test, and deployment tasks using the Jenkins web interface without writing pipeline code.

- 1. Freestyle jobs are created and managed entirely through the Jenkins UI.
- 2. They support code management tools like Git to pull project code.
- 3. Build steps such as compiling, running scripts, or executing tests can be added manually.
- 4. Plugins can be used to extend functionality
- 5. They are best suited for small projects or beginners learning Jenkins.